

Marine Race Arena: A Configurable HoloOcean Benchmark for Underwater Gate Racing and Team-Level Fleet Evaluation

Lorenzo Bacchiani, Andrea Bedei, Roberto Girau and Giovanni Pau
Department of Computer Science and Engineering, University of Bologna, Italy

Abstract—Autonomous racing provides a compact and reproducible setting for evaluating perception, planning and control under time constraints. Ground and aerial robotics have established racing platforms and gate-based competitions, but underwater robotics lacks a comparable benchmark for speed-oriented racing along predefined gate courses. This paper presents Marine Race Arena, a configurable benchmark layer built on HoloOcean with a BlueROV2 vehicle. A single JSON configuration specifies the track, gates, sensors, currents, obstacles and scoring rules. The benchmark strictly separates control from evaluation. Controllers receive only an official observation composed of onboard sensing, acoustic guidance and race progress. An independent referee uses privileged simulator state only for gate validation, penalties, timeouts and scoring. We formalize the benchmark model, controller interface, gate-crossing rules and scoring procedure. We also extend the benchmark to a staggered multi-rover team with per-vehicle referee state and team-level scoring. In HoloOcean, a deterministic rule-based baseline completes the three official clean tracks and is also evaluated under optional disturbance currents. On a deterministic kinematic substrate, a second conservative controller differs measurably from the first in completion time and motion smoothness. A leader-follower controller uses only an optional realistic acoustic channel and reduces a staggered team’s inter-vehicle proximity events to the single-rover level while every rover finishes. HoloOcean diagnostic validation on clean Horseshoe Bay tests this coordination with three rovers, no currents or obstacles, two start gaps and three seeds. Every coordinated run finishes with no inter-vehicle proximity events or gate/world collisions, whereas uncoordinated teams incur many gate/world collisions and sometimes fail to finish. The arena also supports optional obstacles. Marine Race Arena provides a reproducible evaluation contract for developing and comparing autonomous underwater racing controllers, including future learning-based approaches.

Index Terms—underwater robotics, autonomous racing, HoloOcean, BlueROV2, simulation benchmark, referee, multi-robot systems, team-level evaluation

I. INTRODUCTION

Autonomous racing has emerged as an effective research format for evaluating perception, planning, control and real-time decision-making within a single quantitative task under repeatable conditions [1]. In ground robotics, the F1TENTH platform provides standardized vehicles, circuits, metrics and baselines for continuous control and reinforcement learning [2]. At full scale, competitions such as the Abu Dhabi Autonomous Racing League (A2RL) push planning and control toward the limits of vehicle handling in head-to-head racing scenarios [3]. In aerial robotics, gate-based benchmarks and competitions have driven progress from modular perception

and control pipelines [4] to learning-based policies capable of champion-level drone racing [5]. Across these domains, the availability of a well-defined racing task, fixed course geometry, explicit scoring rules and machine-readable outcomes has made autonomy methods easier to compare, reproduce and improve.

Underwater robotics has not yet benefited from an equivalent benchmark for speed-oriented autonomous racing along predefined gate courses. Existing marine simulators provide increasingly mature support for underwater vehicles, sensors and environments, but they mainly target intervention, inspection, navigation or sensor realism rather than racing as an evaluation protocol. In particular, they do not provide a compact underwater counterpart to ground and aerial racing benchmarks: a task defined by ordered gates, timing, penalties, currents, obstacles, independent scoring and reproducible outcomes. This gap matters because underwater racing is not simply aerial or ground racing transferred to a different simulator. Underwater vehicles are slow, strongly damped and affected by persistent currents; global localization is typically unavailable; inertial and Doppler sensing may drift; optical perception is limited in range and degraded by turbidity; acoustic cues are noisy and low bandwidth; and physical contact with gates, obstacles or other vehicles can be difficult to interpret. These properties change both the information available to a controller and the way in which performance should be evaluated.

A central requirement for such a benchmark is therefore a strict separation between autonomy and evaluation. A controller should be evaluated using only information that would plausibly be available onboard, such as local sensing, acoustic target cues and race progress information. Conversely, privileged simulator state, including global pose, exact gate geometry and the true current field, should be reserved for the evaluator. Without this boundary, performance can depend on information that would not be available to a real vehicle, making results difficult to interpret and unfair to compare.

This paper presents Marine Race Arena, a configurable benchmark layer for autonomous underwater gate racing built on HoloOcean [6] and a BlueROV2-class vehicle [7]. Marine Race Arena is not intended to replace hydrodynamic simulators or to propose a new autonomy algorithm. Instead, it defines an evaluation contract on top of an existing underwater simulation backend. A race is specified declaratively through a JSON configuration describing the world, ordered gates,

sensors, currents, obstacles and scoring rules. During execution, controllers receive only an official observation, while an independent referee uses privileged simulator state to validate gate crossings, detect rule violations, assign penalties, manage timeouts and compute the official scores.

The autonomy that drives a rover is supplied by the user: a controller is the rover’s command module, plugged in by implementing a single predefined interface of a few methods (Section V) without modifying the framework. The same mechanism applies to a cooperative fleet, where each rover runs its own controller. This keeps the autonomy stack cleanly separated from the evaluation and lets the benchmark compare arbitrary controllers, including future learning-based ones, under the same official observation.

The contributions of this paper are as follows.

- 1) We introduce a declarative benchmark model for autonomous underwater gate racing, in which the world, ordered gates with $1.5\text{ m} \times 1.5\text{ m}$ apertures, currents, obstacles, sensors and scoring rules are specified in a single JSON configuration. Three official clean tracks are provided.
- 2) We define an official controller observation contract and a HoloOcean race runner that prevent controllers from accessing ground-truth pose, simulator current vectors and hidden gate geometry. A custom controller can be integrated by implementing a minimal three-method interface.
- 3) We design an independent referee that formalizes gate-crossing validation, missed gates, collisions, out-of-bounds and stuck conditions, timeouts, scoring and structured event logging.
- 4) We extend the benchmark to staggered multi-rover execution of one cooperative team. Multiple BlueROV2 rovers follow the same course with independent progress and referee state while contributing to an aggregate team score. The fleet layer provides inter-vehicle proximity diagnostics and an optional realistic inter-rover acoustic channel. Using only the official observation, per-rover race state and this channel, a leader-follower controller keeps the staggered team free of inter-vehicle proximity events while it completes the same gate sequence.
- 5) We provide a reproducible v0.1 validation package with two deterministic gate controllers that differ in timing and behavior. The package includes clean single-rover results on three official tracks, a stable two-rover demonstration and disturbance-current experiments that expose remaining challenges in robust underwater racing. On a deterministic kinematic substrate, the benchmark separates the controllers by completion time and motion smoothness. Leader-follower coordination reduces a staggered team’s inter-vehicle proximity events to the single-rover level while the team finishes. HoloOcean diagnostic validation on clean Horseshoe Bay covers three rovers, no currents or obstacles, two start gaps and three seeds. Every coordinated run finishes with no inter-vehicle proximity events or gate/world collisions,

whereas uncoordinated runs incur many gate/world collisions and sometimes fail to finish.

The remainder of the paper is organized as follows. Section II reviews underwater simulators, autonomous-racing benchmarks and multi-AUV underwater cooperation. Section III formalizes the benchmark model and runtime architecture. Section IV describes the vehicle, the HoloOcean adapter, the sensor suite, the current model and the official observation contract. Section V specifies the controller interface and the rule-based baseline. Section VI formalizes gate validation, the referee state machine and single-rover scoring. Section VII describes staggered fleet execution and team scoring. Section VIII reports the experimental evaluation. Section IX outlines future work. Section X concludes the paper.

II. RELATED WORK

We review related work along three axes: the underwater simulators that provide the physical and sensing substrate; the autonomous-racing benchmarks that define the evaluation format we adapt; and the multi-AUV cooperation and acoustic-communication literature that motivates and bounds the fleet mode. Table I summarizes the positioning of Marine Race Arena against representative systems.

A. Underwater Robotics Simulators

Several simulators provide marine vehicles and sensors. The UUV Simulator offers a Gazebo-based package for underwater intervention and multi-robot scenarios [8]. Stonefish provides custom marine-robotics physics and a wide range of underwater sensors behind a ROS interface [9]. DAVE builds on Gazebo to support acoustic and optical sensing for autonomous underwater vehicles [10]. HoloOcean uses Unreal Engine to render underwater scenes and exposes common marine sensors, sonar, RGB camera, IMU, DVL and depth, for one or more agents [6] and is the simulation backend we build on. These systems are complementary to Marine Race Arena: they simulate vehicles and sensors, whereas Marine Race Arena adds the race task, the official observation contract, the referee, the scoring and the output protocol on top of one of them. None of them ships a gate-racing benchmark with an impartial referee and consequently none enforces an information boundary between the controller and ground-truth state.

B. Autonomous Racing Benchmarks and Competitions

Racing benchmarks are valuable precisely because they expose end-to-end behavior under repeatable, adversarial constraints, as catalogued by the survey of Betz et al. [1]. F1TENTH provides a scaled autonomous car, standardized circuits and an evaluation environment for continuous control and reinforcement learning [2]; it is the closest in spirit to Marine Race Arena, but it targets ground vehicles with planar LiDAR and treats referee logic as only a partial, in-framework concern rather than a separated, first-class contract. At full scale, the A2RL challenge frames racing as multi-agent interaction at the limits of vehicle handling and reports the

TABLE I

POSITIONING OF MARINE RACE ARENA AGAINST REPRESENTATIVE SIMULATORS AND RACING BENCHMARKS. RACING TASK: A BUILT-IN TIMED GATE AND LAP PROTOCOL; INDEP. REFEREE: CONTROLLER-INDEPENDENT SCORING ON PRIVILEGED STATE; OFFICIAL OBS. CONTRACT: AN ENFORCED FILTER EXCLUDING GROUND-TRUTH POSE.

System	Domain	Physics / sensing	Racing task	Multi-robot	Indep. referee	Official obs. contract
UUV Simulator [8]	Underwater	Gazebo, hydrodynamics	no	✓	no	no
Stonefish [9]	Underwater	Custom physics, marine sensors, ROS	no	no	no	no
DAVE [10]	Underwater	Gazebo, acoustic and optical sensing	no	✓	no	no
F1TENTH [2]	Ground	Scaled car, LiDAR, RL and control	✓	partial	partial	✓
AlphaPilot / Swift [4], [5]	Aerial	Quadrotor, vision-based gate racing	✓	no	partial	✓
CARLA [11]	Ground	Unreal Engine, urban sensing, tasks	partial	no	partial	✓
Marine Race Arena (this work)	Underwater	HoloOcean, BlueROV2, acoustic+camera	✓	✓	✓	✓

“no” marks a feature that is not provided as a first-class capability; “partial” marks one that is available but not the primary design contract. Marine Race Arena is the only entry that combines an underwater simulation backend with a gate-racing protocol, an independent referee, multi-rover and team scoring and an enforced official-observation contract.

algorithms of the winning team in the inaugural Abu Dhabi event [3], while the A2RL league also runs a drone-racing format [12]. In aerial robotics, AlphaPilot established gate sequences as a demanding benchmark for fast perception and control [4] and Swift later demonstrated champion-level drone racing using deep reinforcement learning trained against a simulated opponent [5]. CARLA is not a racing benchmark, but it illustrates how an Unreal-based ecosystem can standardize maps, sensors and tasks for driving research [11]. More broadly, standardized environment interfaces such as OpenAI Gym [13] and GPU-parallel robot-learning simulators such as Isaac Gym [14] show how a documented observation-action contract enables systematic comparison of learning-based controllers. Marine Race Arena borrows the gate-racing idea and the documented observation contract from these efforts, but adapts the benchmark to underwater sensing, slow and damped vehicle response, acoustic guidance, current disturbances and a referee whose scoring is computed entirely outside the control loop.

C. Multi-AUV Cooperation and Underwater Communication

Fleet operation underwater is constrained by the physics of the acoustic channel and by uncertain localization. Underwater acoustic links suffer from low bandwidth, long propagation delay, multipath and time-varying attenuation [15], so the assumption of continuous high-bandwidth, low-latency inter-vehicle communication that underpins many terrestrial multi-robot methods does not transfer. Surveys of AUV formation control document how these communication and navigation constraints shape the achievable coordination: Yang et al. analyze formation performance, control and communication capability jointly [16] and Yan et al. review formation-control methods and their dependence on relative-state estimation [17]. Consistent with this literature, Marine Race Arena does not assume shared ground-truth state or high-rate communication between vehicles. In v0.1 the fleet mode is deliberately conservative: it validates multiple HoloOcean agents, staggered releases, independent per-rover progress and team-level scoring while keeping inter-vehicle proximity detection in diagnostic mode by default (off, diagnostic or

penalize). This positions the fleet contribution as an evaluation substrate for future cooperative strategies.

D. Positioning

In contrast to the systems above, Marine Race Arena is the only one that simultaneously provides an underwater simulation backend, a gate-racing protocol with timing, an independent referee that uses privileged state, multi-rover and team-level scoring and an enforced official-observation contract that excludes ground-truth pose from control (Table I). It does not compete with the underwater simulators on physical fidelity, nor with the racing platforms on autonomy performance; it complements both by supplying the missing evaluation contract for underwater gate racing.

III. BENCHMARK MODEL AND SYSTEM ARCHITECTURE

We specify Marine Race Arena as an *evaluation problem* rather than a controller-design problem: the benchmark fixes the world, the task and the scoring and a participant supplies only a policy. This section defines the benchmark instance, the participant interface and the runtime architecture that enforces the separation between the two.

A. Benchmark Instance

Definition 1 (Benchmark instance). A Marine Race Arena instance is the tuple

$$\mathcal{E} = (\mathcal{W}, \mathcal{G}, \mathcal{S}, \mathcal{F}, \mathcal{C}, \mathcal{O}, \mathcal{P}, \mathcal{R}), \quad (1)$$

where:

- $\mathcal{W}$ is the *world*: an axis-aligned bounds box $\mathcal{B} = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] \times [z_{\min}, z_{\max}] \subset \mathbb{R}^3$ (with z positive-up, so the navigable volume satisfies $z < 0$) and the HoloOcean environment in which it is rendered;
- $\mathcal{G} = (g_1, \dots, g_M)$ is the *ordered* gate sequence; each gate is the tuple $g_k = (c_k, n_k, w_k, h_k)$ of center $c_k \in \mathbb{R}^3$, passage-direction vector $n_k \in \mathbb{R}^3$ (nonzero, not necessarily unit length) and inner aperture width w_k and height h_k ;
- $\mathcal{S}$ is the start configuration: a spawn pose and an optional release delay $\tau_i \geq 0$ per participant;

TABLE II
DESIGN REQUIREMENTS FOR THE V0.1 BENCHMARK LAYER.

ID	Requirement
R1	Race configuration is declarative and reproducible from a single set of track JSON files.
R2	Official controller observations exclude ground-truth pose and simulator-only state.
R3	Referee logic is independent of the participant controller and cannot be influenced by it.
R4	Gate dimensions, referee margins and track geometry are not changed to hide failures.
R5	Every run emits structured event logs and machine-readable summaries.
R6	Single-rover behavior is unchanged when fleet mode is disabled.
R7	Fleet mode aggregates a team score while preserving per-rover diagnostics.

- $\mathcal{F}$ is the finish condition: completion of the last gate of the last lap (Section VI);
- $\mathcal{C}$ is the current field $\mathbf{v}_c : \mathbb{R}^3 \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^3$ (Section IV);
- $\mathcal{O}$ is the obstacle set;
- $\mathcal{P} = \{1, \dots, N\}$ is the participant set, each with a vehicle, sensor profile, controller, spawn and release delay;
- $\mathcal{R}$ is the referee configuration: gate-validation parameters, penalty values and scoring rules.

The gate basis is right-handed and derived from the passage direction alone, so the orientation of a gate is unambiguous and independent of any visual rotation (Section VI).

B. Participant Interface

Definition 2 (Policy). A participant i implements a policy

$$\pi_i : o_{i,t} \mapsto u_{i,t}, \quad (2)$$

mapping the official observation $o_{i,t}$ (Section IV) to a normalized, high-level, body-frame command

$$u_{i,t} = [u^{\text{surge}}, u^{\text{sway}}, u^{\text{heave}}, u^{\text{yaw}}]^\top \in [-1, 1]^4. \quad (3)$$

The action of (3) is the simpler, constrained interface; equivalently, a controller may bypass it and command the eight BlueROV2 thrusters directly (a `thrusters` action), a more expressive option, with the choice left to the user. Time is discretized at a fixed control timestep Δt , with $t_k = k \Delta t$. The runner may clamp $u_{i,t}$ for actuator and depth safety; in official mode the policy reads only the official observation $o_{i,t}$, whose contract excludes privileged state (Section IV).

C. Design Requirements

The release-candidate design follows the seven requirements in Table II. They encode the central commitment of the benchmark, that scoring is independent of and not improvable by, the participant and the secondary commitment that enabling fleet mode must not silently change single-rover behavior.

D. Runtime Architecture

Fig. 1 shows the runtime architecture and the modules that realize it. The configuration loader parses and validates the track JSON into a typed configuration; the arena builder instantiates gates, beacons, the current field, obstacles and bounds. The race runner drives the discrete-time loop. On the simulation side, the adapter handles reset, actions, ticking and sensor extraction. The fallback option is a deterministic kinematic adapter that implements the same interface for plumbing tests without the engine. On the control side, the runner assembles the official observation and passes it to the controller, which returns a command. On the scoring side, the referee consumes privileged simulator state, validates progress and emits structured events and summaries. The dashed boundary in Fig. 1 marks the information contract: nodes inside it see only the official observation, while ground-truth state is confined to the referee. This separation keeps participant controllers from influencing scoring logic or accessing privileged evaluation state.

IV. VEHICLE, SIMULATOR AND SENSING

This section describes the simulation backend, the vehicle and its sensors, the acoustic and current models, the declarative track configuration and the official observation contract that the referee enforces.

A. Simulator and Vehicle

The reference adapter uses HoloOcean [6], an Unreal-Engine underwater simulator, with a BlueROV2-class agent [7] in direct eight-thruster mode (HoloOcean `control_scheme` 0). The simulator advances at 30 physics ticks per second. The benchmark uses a control timestep of $\Delta t = 0.033$ s, so each control step corresponds to one physics tick and the effective control rate is ≈ 30 Hz. The adapter is replaceable. Its fallback option is a deterministic kinematic adapter that implements the same interface for unit and plumbing tests. It integrates a damped point-vehicle model (maximum linear speed 1.25 m/s, maximum yaw rate $65^\circ/\text{s}$) and adds the current as a world-frame velocity. This lets the referee, scoring and fleet logic be tested deterministically without the engine.

A high-level command $u = [u^{\text{surge}}, u^{\text{sway}}, u^{\text{heave}}, u^{\text{yaw}}]^\top$ is mapped to the eight BlueROV2 thrusters by a fixed mixing matrix. The four vertical thrusters receive u^{heave} ; the four horizontal thrusters receive

$$\tau_{1-4} = s_\tau \begin{bmatrix} u^{\text{surge}} + u^{\text{sway}} + \kappa u^{\text{yaw}} \\ u^{\text{surge}} - u^{\text{sway}} - \kappa u^{\text{yaw}} \\ u^{\text{surge}} + u^{\text{sway}} - \kappa u^{\text{yaw}} \\ u^{\text{surge}} - u^{\text{sway}} + \kappa u^{\text{yaw}} \end{bmatrix}, \quad (4)$$

with yaw-mixing gain $\kappa = 0.35$ and thruster scale s_τ , each component clamped to the thruster limit. A controller may instead return the eight thruster values directly; the high-level interface is the default because it is simpler and vehicle-agnostic (a controller expresses intended body-frame motion while the framework handles the mixing and depth-safety),

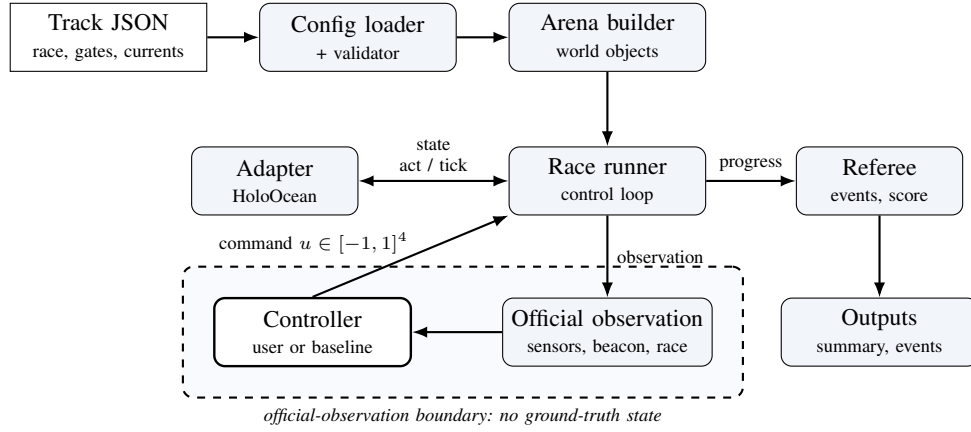


Fig. 1. Runtime architecture. Declarative configuration builds the arena, the race runner drives the control loop and the independent referee scores it. Inside the dashed official-observation boundary the controller sees only the filtered observation and returns a command; ground-truth state stays on the referee side.

TABLE III
ONBOARD SENSOR SUITE OF THE BLUEROV2.

Sensor	Rate	Output and configuration
Depth	30 Hz	Scalar depth (m); noise $\sigma = 0$.
IMU	30 Hz	Linear acceleration and angular rate, with bias terms.
DVL	15 Hz	Body-frame velocity (m/s); beam elevation 22.5° , max range 50 m.
Front camera	30 Hz	RGB image, 640×480 , 90° field of view.
Velocity	30 Hz	World-frame linear velocity (m/s).
Collision	30 Hz	Boolean contact flag.

Derived fields: heading ψ (deg) and depth (m) from the pose used by the referee; the true environment current vector is stripped from official observations entirely and is available only as non-official diagnostic telemetry.

whereas direct thruster control is available when finer actuator authority is wanted.

B. Sensor Suite

Table III lists the onboard sensors. The official profile combines a forward RGB camera, a depth sensor, an IMU, a DVL, a world-velocity sensor and a contact sensor. Ground-truth pose, location, rotation and dynamics sensors exist in the simulator and are used internally by the referee, but are removed from the controller-visible observation in official mode (Section IV-F).

C. Acoustic Beacon Model

Let $(\hat{n}, \hat{r}_g, \hat{s}_g)$ be the right-handed gate frame of (11), with $\hat{n}$ the passage normal and $\hat{s}_g$ the gate-up axis. Each gate carries one beacon, placed at $b_g = c_g + \delta_n \hat{n} + \delta_r \hat{r}_g + \delta_s \hat{s}_g$ relative to the gate center c_g (default offset $(0, 0, 0.35)$ m along $\hat{s}_g$). For a receiver at position p_r with heading ψ , let $\delta = b_g - p_r$ and

$\rho = \|\delta\|$. The beacon reports range, a yaw-relative bearing, an elevation and a signal strength:

$$\rho = \|\delta\|, \quad \beta = \text{wrap}(\text{atan2}(\delta_y, \delta_x) - \psi), \quad (5)$$

$$\varepsilon = \text{atan2}(\delta_z, \sqrt{\delta_x^2 + \delta_y^2}), \quad \text{SS} = \max\left(0, 1 - \frac{\rho}{\rho_{\max}}\right),$$

where $\rho_{\max}$ is the beacon range and $\text{wrap}(\cdot)$ maps to $(-180^\circ, 180^\circ]$. A reading is invalid beyond $\rho_{\max}$ or when a Bernoulli dropout fires. In the noisy acoustic mode, independent Gaussian noise $\mathcal{N}(0, \sigma_b)$ is added to β , ε and ρ ; the official tracks use $\sigma_b \in \{0.2, 0.6\}$ and dropout probabilities up to 0.04. Only the beacon of the current target gate is active in the default *active-when-target* mode, so the beacon provides a target cue but not a map of the whole course.

D. Current Field Model

The current field is a sum of analytic primitives evaluated at the vehicle position p and time t ,

$$\mathbf{v}_c(p, t) = \sum_j \mathbf{v}_j(p, t), \quad (6)$$

where each primitive is one of the following. A *constant* term contributes a fixed velocity $\mathbf{v}_0$. A *localized jet* centered at c_j with radius R_j contributes, for $\|p - c_j\| \leq R_j$,

$$\mathbf{v}_j = \mathbf{v}_0 \omega(d), \quad \omega(d) = \begin{cases} \max(0, 1 - \frac{d}{R_j}) & \text{linear,} \\ \exp(-\frac{d^2}{2\sigma^2}) & \text{Gaussian,} \end{cases} \quad (7)$$

with $d = \|p - c_j\|$ and $\sigma = \max(R_j/2, 10^{-6})$ and zero outside R_j . A *sinusoidal* term modulates one axis, $v_{j,\text{axis}}(t) = v_{0,\text{axis}} + A \sin(2\pi f t + \phi)$. A *vortex* of radius R_j , tangential speed V_t and vertical speed V_z contributes, with $r = \sqrt{\Delta x^2 + \Delta y^2}$ and tangent $\hat{\theta} = (-\Delta y, \Delta x)/r$,

$$\mathbf{v}_j = (\hat{\theta}_x V_t \omega(r), \hat{\theta}_y V_t \omega(r), V_z \omega(r)). \quad (8)$$

The provided `strong` profile superposes a constant set-flow of $[0.75, 1.05, 0]$ m/s, two Gaussian jets, a vortex and a vertical

sinusoid; the medium profile scales these magnitudes by 0.5. The current physically perturbs the vehicle when the HoloOcean build exposes ocean-current coupling; otherwise the field is exposed as telemetry only and the deterministic kinematic adapter always couples it.

E. Declarative Track Configuration

A track is a self-contained JSON file. Its principal sections are `race` (name, benchmark task, timing mode, laps and duration), `world` (bounds and environment), `start` and `finish`, `track` and `gates` (the ordered gate sequence with centers, passage directions and aperture sizes), `beacon`, `currents`, `obstacles`, `participants` (vehicle, spawn, sensors, controller and optional `start_delay_s`) and `referee` (gate validation, penalties and scoring). A named `benchmark_task` (`clean_gate`, `obstacle_gate`, `current_gate` or `multi_rov`) selects a preset overlay. Listing 1 shows a representative excerpt.

Listing 1. Representative excerpt of a track configuration.

```
"track": {
  "gate_inner_size_m": [1.5, 1.5],
  "gate_sequence": ["G01", "G02", "..."]
},
"gates": [
  { "id": "G01", "center": [4.0, -2.0, -3.5],
    "passage_direction": [1.0, 0.0, 0.0] }
],
"beacon": {
  "mode": "active_when_target", "range_m": 90.0
},
"participants": [
  { "id": "bluerov2_01", "vehicle": "BlueROV2",
    "controller": "rule_gate_baseline",
    "start_delay_s": 0.0 }
],
"referee": {
  "penalties": { "minor_collision_s": 5.0 },
  "gate_validation": {
    "vehicle_clearance_margin_m": 0.1 }
}
```

The three official v0.1 tracks are summarized in Table IV; all use $1.5\text{ m} \times 1.5\text{ m}$ apertures that are never enlarged for validation (requirement R4).

F. Official Observation Contract

The controller `step` function receives a dictionary with the fields in Table V. In official mode the runner removes pose, location, rotation, dynamics, exact-position, debug ground-truth and exact-gate-geometry fields, then keeps only the configured allow-list of sensor names; numeric arrays are made JSON-safe while image arrays are preserved. The true environment current vector is privileged disturbance state: it is stripped from official observations unconditionally (this strip cannot be re-enabled by an allow-list entry) and remains available only as diagnostic telemetry in non-official runs. A controller must therefore infer current effects from its onboard sensing (e.g. the DVL or velocity-sensor residual), not read the current directly. The intended information contract is acoustic target guidance, onboard sensing and race progress, not global state.

When the optional inter-rover acoustic channel is enabled (Section VII), the observation additionally carries a `comms` inbox whose messages are authored by other controllers and therefore contain only information those controllers may legally observe.

V. CONTROLLER INTERFACE AND RULE BASELINE

A central usability goal of Marine Race Arena is that a custom autonomy stack is integrated by implementing a small interface, without touching the runner, the referee or the adapter. This section specifies that interface, the dynamic loading mechanism, the action mapping and the reference baseline whose results we report.

A. Controller Interface

A controller implements three methods: `reset(race_info)`, called once before the run with static race metadata (timing mode, laps, official mode, bounds and the initial target-gate id); `step(observation)`, called every control tick with the official observation of (2) and returning the command of (3); and `close()`, called at teardown. The contract is duck-typed: any object exposing these three callables is accepted, so a participant need not depend on framework base classes. A manual controller may raise a dedicated stop exception from `step` to end a run cleanly and any other exception is caught, recorded and treated as a zero command so that one faulty controller cannot crash an evaluation. Listing 2 shows a minimal example.

Listing 2. Minimal custom controller.

```
class MyController:
    def reset(self, race_info):
        self.target = race_info.get(
            "initial_target_gate_id")

    def step(self, observation):
        beacon = observation.get("beacon", {})
        # ... map beacon/sensors to a command ...
        return {"surge": 0.3, "sway": 0.0,
                "heave": 0.0, "yaw": 0.0}

    def close(self):
        pass
```

The high-level command fields are listed in Table VI; values are normalized to $[-1, 1]$ and clamped by the framework before the action mapping. In fleet mode a controller may additionally return a small message payload alongside its command and read incoming teammate messages from the observation, using the optional inter-rover acoustic channel of Section VII.

B. Dynamic Loading

A controller is selected at run time in one of three ways: the official built-in rule baseline `rule_gate_baseline`; a dotted module path with an explicit class; or a file path together with `--controller-class`, which is imported dynamically. For inspection and data collection, the runner can also be launched with the manual keyboard or pygame

TABLE IV
OFFICIAL V0.1 TRACK CONFIGURATIONS.

Track file	Track name	Gates	Laps	Total gates	Length (m)	Intended use
marine_race_horseshoe_bay.json	Marine Race Horseshoe Bay	12	1	12	93.8	Clean-gate horseshoe baseline
marine_race_vertical_serpent.json	Marine Race Vertical Serpent	17	1	17	118.3	Vertical and slalom gate sequencing
marine_race_mixed_endurance.json	Marine Race Mixed Endurance	22	1	22	206.3	Longer endurance and current-oriented layout

TABLE V
TOP-LEVEL FIELDS OF THE OFFICIAL OBSERVATION.

Field	Meaning
time_s	Race time in seconds.
participant_id	Identifier of the vehicle receiving the observation.
sensors	Filtered onboard sensor dictionary (camera, depth, IMU, DVL, velocity, collision and the derived heading and depth).
beacon	Acoustic message: validity, target gate id, bearing, elevation, range, signal strength and mode.
race	Participant-visible progress: status, lap, completed gates, target gate id, target sequence index and official-timing status.
comms	Present only when the optional inter-rover acoustic channel is enabled: an <code>inbox</code> of messages received from teammates, each with the sender id, the controller-authored payload and send and receive timestamps (Section VII).

TABLE VI
HIGH-LEVEL CONTROLLER COMMAND FIELDS OF (3).

Field	Interpretation
surge	Longitudinal body-frame thrust; positive moves forward.
sway	Lateral body-frame thrust; sign follows the adapter mixing of (4).
heave	Vertical thrust; additionally constrained by depth-safety logic near the bounds.
yaw	Yaw-rate command; sign follows the adapter mixing of (4).

controllers. This lets an external policy be evaluated without modifying the package:

```
python -m marine_race_arena.scripts.run_marine_race \
--track ../marine_race_horseshoe_bay.json \
--benchmark-task clean_gate \
--participant-controller path/to/my_controller.py \
--controller-class MyController \
--adapter holoocean --official --headless
```

C. Action Mapping

The framework clamps the returned command to $[-1, 1]^4$ and applies a depth-safety rule that suppresses heave that would drive the vehicle past $z_{\min}$ or $z_{\max}$, then maps it to thrusters via (4). A controller may alternatively return raw thruster values; the high-level interface is the simpler default.

D. Baseline Based on Rules

The reference controller is deliberately interpretable rather than optimal. Its role is to be a deterministic policy that, using only official observations, completes the clean tracks and exposes failures under currents, obstacles or dense fleet interaction. Let β and ρ be the beacon bearing and range, z the depth and z^* the target depth implied by the gate. The policy is organized into the four explicit components summarized in Fig. 2.

- *Bearing alignment and surge gating.* The controller aligns the rover before allowing sustained forward motion. Yaw tracks the beacon bearing through a clamped proportional law with a small deadband,

$$u^{\text{yaw}} = \text{clamp}(k_{\psi} \beta, -\bar{u}_{\text{yaw}}, \bar{u}_{\text{yaw}}), \quad \bar{u}_{\text{yaw}} = 0.16, \quad (9)$$

and surge is gated by alignment: the vehicle turns in place when $|\beta| \geq 34^\circ$, surges at a reduced rate for $24^\circ \leq |\beta| < 34^\circ$ and otherwise drives forward up to a conservative cap $\bar{u}_{\text{surge}} = 0.46$ that scales with range and with $\cos \beta$.

- *Depth hold.* The controller keeps the rover close to the target gate depth. A PI law on depth error $e_z = z^* - z$ produces a depth-keeping heave,

$$u_{\text{depth}}^{\text{heave}} = \text{clamp}\left(k_p e_z + k_i \int e_z dt\right), \quad (10)$$

$$(k_p, k_i) = (0.20, 0.08),$$

which is blended with the beacon-elevation heave as $u^{\text{heave}} = 0.70 u_{\text{beacon}}^{\text{heave}} + 0.30 u_{\text{depth}}^{\text{heave}}$.

- *Visual centering.* This component refines the acoustic approach in the final meters. When the front camera detects a gate, normalized image errors $(c_x, c_y) \in [-1, 1]^2$ nudge sway and yaw by $-c_x$ and heave by $-c_y$ above thresholds $(0.13, 0.30)$.
- *Exit hold and command smoothing.* This component stabilizes the command after a gate crossing. After each gate the controller holds an exit heading for a fixed number of steps to clear the structure and all commands are low-pass filtered, $u_t = (1 - \alpha) u_{t-1} + \alpha u_t^{\text{raw}}$ with $\alpha = 0.45$ and rate-limited per axis to avoid abrupt thrust changes.

E. A Second Controller for Comparison

A benchmark must separate controllers that behave differently. Alongside the camera-assisted rule baseline, the framework therefore provides two beacon-only controllers. The first, `acoustic_baseline`, is a fast deterministic controller that uses explicit approach, transit and exit phases. It servos only on the acoustic beacon and uses no camera. The second,

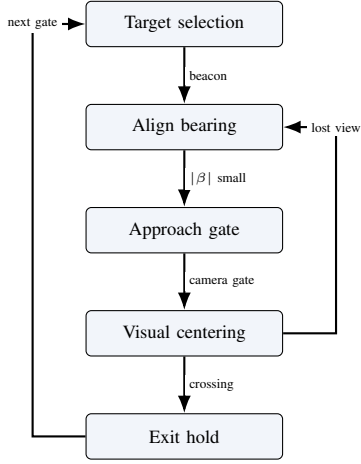


Fig. 2. Simplified state logic of the rule-based baseline: target selection, bearing alignment, gate approach, visual centering and exit hold before advancing to the next gate.

`smooth_gate_baseline`, is a conservative retuning at a different operating point. It uses a lower surge cap, gentler braking, softer yaw gains, a longer exit settle and tighter smoothing and rate limits. It also enters the transit phase from slightly farther out but within a tighter bearing window. Both controllers satisfy the official observation contract and can run without a camera. `smooth_gate_baseline` is intended to complete the same gates more slowly with markedly lower-jerk motion, giving the benchmark a distinct point in the time-smoothness space (Section VIII). Neither is proposed as the better racer; they demonstrate that legal algorithms can produce different measurable times and behavior under the same contract.

VI. REFEREE, GATE VALIDATION AND SCORING

The referee evaluates progress independently of the controller (requirement R3) using privileged ground-truth state. This section formalizes the gate-crossing test, the referee state machine and its arbitration order, the penalty model and the single-rover score.

A. Gate Frame and Crossing

Each gate's orientation derives solely from its passage direction n_g , giving a right-handed frame

$$\hat{n} = \frac{n_g}{\|n_g\|}, \quad \hat{r}_g = \frac{\hat{z} \times \hat{n}}{\|\hat{z} \times \hat{n}\|}, \quad \hat{s}_g = \hat{n} \times \hat{r}_g, \quad (11)$$

with world-up $\hat{z} = (0, 0, 1)$; if the passage direction is vertical ($\|\hat{z} \times \hat{n}\| \approx 0$), $\hat{r}_g$ falls back to the world $\hat{x}$ axis so the frame stays well defined. For consecutive referee-side positions p_{t-1}, p_t , the signed distances to the gate plane are

$$d_{t-1} = \hat{n}^\top (p_{t-1} - c_g), \quad d_t = \hat{n}^\top (p_t - c_g). \quad (12)$$

A candidate crossing requires a sign change in the *correct direction*,

$$d_{t-1} \leq 0 \leq d_t \wedge (p_t - p_{t-1})^\top \hat{n} > 0, \quad (13)$$

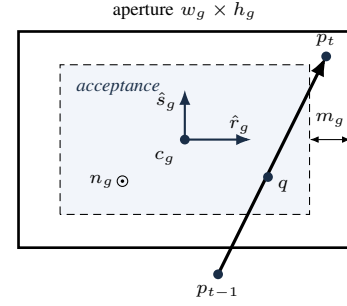


Fig. 3. Gate-crossing validation. A valid crossing pierces the gate plane in the normal direction ((13)) at a point q ((14)) inside the aperture shrunk on every side by the safety margin m_g ((15)); the frame $(\hat{r}_g, \hat{s}_g, n_g)$ follows the passage direction.

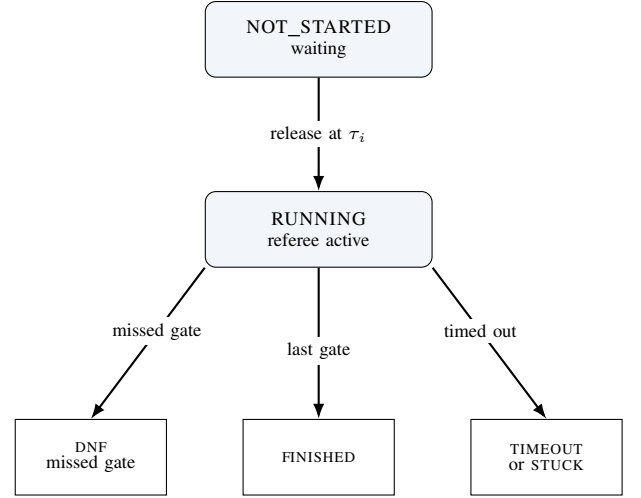


Fig. 4. Participant lifecycle. A rover is released to RUNNING at its start delay τ_i (a single rover is the case $\tau_i = 0$); while waiting it receives a zero command and is not stepped. A running rover reaches a terminal state by finishing, missing a gate (DNF) or timing out.

i.e. travel from the negative to the positive side along the normal. The intersection of the segment with the plane is

$$\lambda = \frac{-d_{t-1}}{d_t - d_{t-1}}, \quad q = p_{t-1} + \lambda(p_t - p_{t-1}), \quad \lambda \in [0, 1], \quad (14)$$

and the crossing is accepted only when q lies inside the aperture, shrunk by the configured safety margin m_g ,

$$|\hat{r}_g^\top (q - c_g)| \leq \frac{w_g}{2} - m_g, \quad |\hat{s}_g^\top (q - c_g)| \leq \frac{h_g}{2} - m_g. \quad (15)$$

Gates must be passed *in sequence*: the referee tests only the expected gate g_ℓ , advancing the index $\ell \leftarrow \ell + 1$ on a valid crossing and incrementing the lap when the sequence completes. A crossing of a *non-target* gate aperture is a missed-gate event; a sign change in the wrong direction is logged but neither penalized nor counted as progress. Fig. 3 illustrates the test and Fig. 4 the participant lifecycle.

B. Referee State Machine and Arbitration

Each participant occupies one of the states NOT_STARTED, RUNNING, FINISHED, DNF, TIMEOUT, STUCK, MAN-

TABLE VII
REFEREE EVENTS, DEFAULT PENALTIES AND WHICH TERMINATE THE RUN. ALL VALUES ARE CONFIGURABLE IN THE REFEREE BLOCK.

Event	Penalty	Terminal
Gate, bar or generic collision	5.0 s each	no
Obstacle collision	5.0 s each [†]	no
Out of bounds	10.0 s each	no
Stuck (soft)	15.0 s once	no
Missed gate	n/a	yes (DNF)
Timeout	n/a	yes
Gate-timeout stuck	n/a	yes
Inter-vehicle proximity event	5.0 s per pair	no (team-level)

[†]Per-obstacle value if specified, else the collision default. A per-event cooldown (1.0 s) prevents a single sustained contact from being charged repeatedly.

UAL_STOP or CONTROLLER_ERROR; all but the first two are terminal. A participant is released to RUNNING when the race clock reaches its start delay (Section VII). On each tick of a running participant, events are evaluated in a fixed priority order that defines the arbitration: (1) a controller error transitions to CONTROLLER_ERROR and returns; (2) out-of-bounds; (3) generic collision; (4) obstacle collision; (5) stuck accumulation; (6) duration timeout; (7) gate progression. Steps (2) to (5) accrue penalties without terminating; only (1), (6), a missed gate in (7) or an external gate-timeout produce a terminal state. To avoid double counting, an obstacle-collision tick suppresses the generic collision flag and each event class has an independent cooldown.

C. Penalties and Termination

Penalties accumulate into a per-participant total P_i ; the defaults applied in v0.1 are listed in Table VII. The non-trivial conditions are formalized as follows. The instantaneous speed is $s_t = \|p_t - p_{t-1}\|/\Delta t$; a stuck accumulator integrates low-speed time,

$$T^{\text{stuck}} \leftarrow \begin{cases} T^{\text{stuck}} + \Delta t, & s_t < v_{\text{stuck}} \\ 0, & \text{otherwise,} \end{cases} \quad (16)$$

and a one-time soft penalty is charged when $T^{\text{stuck}} \geq T_{\text{max}}^{\text{stuck}}$, with $v_{\text{stuck}} = 0.02$ m/s and $T_{\text{max}}^{\text{stuck}} = 45$ s. An out-of-bounds event is $p_t \notin \mathcal{B}$. A duration timeout, disabled by default, fires when $t - t_{\text{green}} > T_{\text{max}}$. A missed gate sets DNF when `missed_gate_dnf` is enabled (the default), but carries no time penalty, so a disqualifying error is never cheaper than a slow-but-legal lap.

D. Single-Rover Score and Ranking

For a finished participant the official time T_i^{official} is measured between the first and last gate (or green-to-finish, per the timing mode) and the score is the penalized time

$$T_i^{\text{pen}} = T_i^{\text{official}} + P_i. \quad (17)$$

Ranking places finished participants before unfinished ones and is defined by the ascending lexicographic key

$$\kappa_i = (\bar{f}_i, T_i^{\text{pen}}, -G_i^{\text{done}}, n_i^{\text{coll}}, P_i, \text{dist}_i), \quad (18)$$

where $\bar{f}_i = 0$ for finished and 1 otherwise, G_i^{done} is the number of completed gates, n_i^{coll} the collision count and dist_i the distance to the next gate. Thus finished rovers sort by penalized time, while unfinished rovers sort by progress, then fewer collisions, then fewer penalties, then proximity to the next gate, so partial runs remain informative rather than being discarded.

E. Event Logging

Every run emits a line-delimited event log and a machine-readable summary (requirement R5). Events include `gate_passed`, `wrong_direction`, `collision`, `obstacle_collision`, `out_of_bounds`, `stuck`, `race_finish` and `dnf`, each timestamped and attributed to a participant. The summary records, per participant, the status, official and penalized times, completed gates and counts of collisions, out-of-bounds, missed gates and stuck events, together with the final ranking; in fleet mode it additionally carries the team summary of Section VII.

VII. FLEET AND TEAM EVALUATION

Fleet mode runs several rovers as a single cooperative team through the same circuit, so the benchmark can measure how quickly the team completes the course, rather than pitting them as competitors against each other (requirement R7). The benchmark models one team only: all rovers share a single simulated world and a single referee, each keeps an independent progress and scoring state and enabling fleet mode does not alter single-rover behavior (R6).

A. Staggered Release and Spawn Geometry

Each participant i is assigned a release delay $\tau_i = (i-1)\Delta\tau$ for a start gap $\Delta\tau$, so rovers enter the course at $0, \Delta\tau, 2\Delta\tau, \dots$. A waiting rover is sent a zero command, its controller `step` is *not* called and neither stuck nor gate-timeout timers accumulate; at release the referee logs a `participant_released` event and resets the rover's progress timers, so a delayed start is never misclassified as stuck or timed out. Spawn poses are laterally separated about the base spawn to reduce launch interference: the k -th rover is offset by

$$o_k = (-1)^k \left\lceil \frac{k}{2} \right\rceil s \quad (19)$$

along the right axis $(-\sin\psi_0, \cos\psi_0)$ of the base yaw ψ_0 , giving the alternating pattern $0, -s, +s, -2s, +2s, \dots$ for spacing s ; a candidate that would fall outside the world bounds $\mathcal{B}$ is scaled inward until admissible.

B. Inter-Vehicle Proximity Diagnostics

Inter-vehicle proximity events are detected on the referee side and are never exposed to controllers. For two running rovers a, b the detector applies a separable cylinder test

$$\sqrt{(x_a - x_b)^2 + (y_a - y_b)^2} \leq r_{xy} \wedge |z_a - z_b| \leq r_z, \quad (20)$$

with hysteresis (a pair is released only when the horizontal distance exceeds a release threshold r_{rel} or the vertical distance exceeds r_z) and a refractory cooldown to debounce sustained proximity. The v0.1 thresholds are $r_{xy} = 0.8$ m, $r_z = 0.75$ m, $r_{\text{rel}} = 1.05$ m and a 1.0 s cooldown. Three modes are provided: `off` disables detection; `diagnostic` logs inter-vehicle proximity events without changing any score; `penalize` also charges one team-level penalty per pair event. Diagnostic mode is the default.

C. Inter-Rover Acoustic Communication

Because the fleet is a single cooperative team, Marine Race Arena provides an optional channel so that rovers can share information while running the course. It is off by default and changes nothing for single-rover runs (R6). A controller transmits by returning a small message payload alongside its command and receives an `inbox` of teammates' messages in its observation; what a message contains and how it is used are entirely the user's responsibility, so the framework provides only the transport.

To stay faithful to the medium rather than model a perfect radio link, the channel reproduces the constraints of the underwater acoustic channel of Section II. A message broadcast by rover i reaches a teammate j only if the two are within a maximum range R_{max} , after a range-dependent delay

$$\tau_{ij} = \frac{d_{ij}}{c} + \tau_{\text{proc}}, \quad (21)$$

where d_{ij} is the inter-rover distance, c the speed of sound and τ_{proc} a fixed processing delay; each delivery is additionally dropped with probability p_{loss} , payloads are capped to a small size to model the low bandwidth and a rover may not transmit faster than a minimum interval (a half-duplex rate limit). Packet loss is drawn from a seeded generator, so a run stays reproducible.

The channel preserves the official observation contract. A payload is authored by the sending controller, which sees only its official observation, so it can carry only information that controller may legally observe; the framework uses the true rover positions solely to compute the channel physics (d_{ij} , τ_{ij} , range and loss) and never injects pose, geometry or other privileged state into a delivered message. Inter-rover communication is therefore the substrate for the leader-follower coordination of Section VII-D.

D. Leader-Follower Coordination

A staggered team of identical controllers stays separated in time, but a heterogeneous team does not: a faster follower released behind a slower leader catches it and they bunch up at the gates, where every rover converges on the same aperture

center. We therefore provide a coordination controller that keeps the team a spaced convoy using only legal information and wraps any legal gate-passing controller rather than replacing it, so coordination and gate navigation stay separable.

The controller is a thin, fully distributed layer over the acoustic channel of (21). Within the channel's half-duplex rate limit, each rover periodically broadcasts a small heartbeat carrying its completed-gate count g_i , target-gate index and status. It builds a team roster from the sender ids in its inbox. Rover j identifies its *predecessor* $\text{pred}(j)$ as the audible teammate that started just ahead of it, that is the largest index $i < j$ heard within a short freshness window; the frontmost rover has no predecessor and races freely, so the first rover acts as the leader. A follower yields while its predecessor is fewer than Δg completed gates ahead,

$$\text{yield}_j \iff g_{\text{pred}(j)} - g_j < \Delta g, \quad (22)$$

and otherwise runs its wrapped base controller unchanged. While yielding it holds horizontal position with a small active hover, so it neither advances toward the vehicle ahead nor freezes dead in the water. A predecessor that has left the course or gone silent beyond the freshness window stops blocking. With the channel disabled, the controller has no roster and reduces to the base controller.

The design turns a legal progress signal into physical separation. Because the gates are ordered and spatially separated along the course, holding a two-gate progress lead ($\Delta g = 2$, the default) keeps at least one full gate of physical spacing between consecutive rovers, which is far larger than the referee proximity thresholds of (20). A one-gate lead is insufficient, because a follower can reach a gate at the moment its predecessor has only just cleared it; we confirm this failure of $\Delta g = 1$ empirically in Section VIII. The policy reads only the official observation, per-rover race state and controller-authored messages and delegates all motion to a legal base controller, so it stays inside the official observation contract (R6).

E. Team-Level Score

Only the `team_summary` is the official fleet result; per-rover rows are diagnostics. For a team $\mathcal{P}$ of N rovers with G_{exp} expected gates each,

$$G_{\text{team}}^{\text{exp}} = N G_{\text{exp}}, \quad G_{\text{team}}^{\text{done}} = \sum_{i \in \mathcal{P}} G_i^{\text{done}}. \quad (23)$$

The team finishes only if every rover finishes,

$$\text{finished}_{\text{team}} = \bigwedge_{i \in \mathcal{P}} \text{finished}_i, \quad (24)$$

in which case the team elapsed time spans the first release to the last finish,

$$T_{\text{team}} = \max_i T_i^{\text{finish}} - \min_i T_i^{\text{release}}, \quad (25)$$

and the penalized team score aggregates every per-rover penalty together with the team-level inter-vehicle term,

$$T_{\text{team}}^{\text{pen}} = T_{\text{team}} + \sum_{i \in \mathcal{P}} P_i + P_{\text{ivc}}, \quad (26)$$

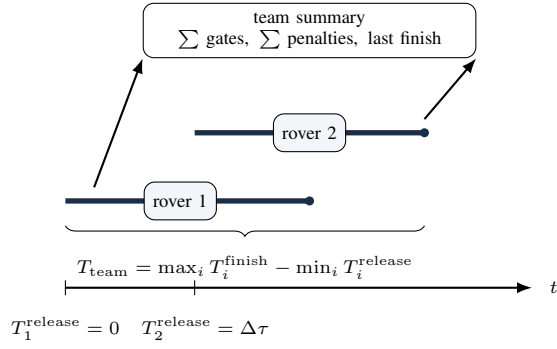


Fig. 5. Team-level evaluation of a staggered two-rover fleet: the team elapsed time spans the earliest release to the latest finish ((25)) and the team summary aggregates gates and penalties ((23) to (26)). The rovers form one cooperative team, not competitors.

where P_i is the per-rover penalty total of (17) and P_{ivc} counts each inter-vehicle proximity event once per pair, not once per rover. Fig. 5 illustrates the aggregation.

VIII. EXPERIMENTAL EVALUATION

This evaluation does not seek an optimal underwater racing controller. It tests whether the benchmark, official observation contract, referee and team scoring behave as specified. It also reports where the reference baseline fails.

A. Protocol and Metrics

The evaluation has three experiment groups. First, the HoloOcean clean/current validation covers the three official clean tracks, the two-rover smoke demonstration on Horseshoe Bay and the medium and strong current profiles on Horseshoe Bay. These experiments use the HoloOcean adapter with the BlueROV2 agent, a control timestep of $\Delta t = 0.033$ s, official mode and the unmodified official gate apertures (requirement R4). Because HoloOcean execution is not bit-deterministic, each condition in Table VIII is evaluated over five seeds.

Second, the deterministic fallback controller/fleet comparison uses the deterministic kinematic adapter on Horseshoe Bay in official mode. It compares two single-rover controllers and three-, four- and five-rover fleets with and without leader-follower coordination. These results are exactly reproducible. Inter-vehicle proximity events on this substrate represent geometric proximity, not contact dynamics.

Third, the HoloOcean diagnostic validation of coordination returns to the HoloOcean adapter. Its scope is only the clean Horseshoe Bay track with three rovers, no currents or obstacles, two start gaps (8.0 and 0.0 s) and three seeds. It compares the same team with and without leader-follower coordination in diagnostic inter-vehicle mode.

The HoloOcean clean/current metrics are course completion, completed gates, official time, penalized time from (17), gate/world collisions, out-of-bounds events and stuck events. The deterministic comparison also reports command smoothness, inter-vehicle proximity events and team penalized time.

The HoloOcean diagnostic validation of coordination reports team completion, inter-vehicle proximity events, gate/world collisions and team times. Completed-gate and gate/world collision statistics include all seeds. Time statistics include finished runs only and are omitted when no seed finishes.

B. Single-Rover Clean Tracks

Table VIII reports the clean-track results over five seeds. The official rule baseline completes all three courses in every seed: 12/12, 17/17 and 22/22 gates, or 100% completion. No run records an out-of-bounds or stuck event. Mean gate/world collisions remain below one per run: 0.6 ± 0.5 , 0.2 ± 0.4 and 0.8 ± 0.8 on Horseshoe Bay, Vertical Serpent and Mixed Endurance. Official times are stable across seeds at 232.4 ± 7.5 s, 259.7 ± 27.8 s and 540.6 ± 7.7 s. These results establish that the official clean tasks are feasible and reproducible under the official observation contract.

C. Staggered Fleet and Team Scoring

The bottom block of Table VIII reports the stable two-rover staggered demonstration on Horseshoe Bay over five seeds. Both rovers finish 12/12 gates in every seed. The first rover finishes in 230.7 ± 5.4 s with 1.4 ± 0.5 gate/world collisions. The second finishes in 236.5 ± 1.4 s with 1.2 ± 0.4 gate/world collisions. The team completes 24/24 gates in every seed. Its elapsed time is 340.7 ± 1.4 s from first release to last finish, and its penalized time is 353.7 ± 3.2 s. No inter-vehicle proximity events are recorded because the 90 s start gap deliberately separates the rovers. The run therefore tests multi-rover spawning, staggered release, independent per-rover referee state, ranking and team aggregation without physical interaction. It is a low-contact smoke demonstration of the fleet and team machinery.

D. Behavior Under Currents

Clean-track performance degrades as current strength increases. On Horseshoe Bay, the rule baseline finishes only 3/5 seeds under the medium profile. Across all five seeds, it completes 10.8 ± 1.5 of 12 gates and accumulates 82.6 ± 74.1 gate/world collisions. Among the three finished seeds, official time is 315.0 ± 42.3 s and penalized time is 428.3 ± 98.5 s. Under the strong profile, the baseline never finishes. Every seed reaches only 3/12 gates within the time limit and accumulates 11.0 ± 1.1 gate/world collisions. These results quantify the degradation under the official observation contract with the gates, margins and current profiles unchanged (requirement R4). Designing controllers that reject the current remains future work (Section IX).

E. Controller Comparison and Fleet Coordination

This experiment asks whether the benchmark separates controllers and whether coordination controls fleet interaction. It uses the engine-free deterministic kinematic adapter on Horseshoe Bay in official mode. The official gate apertures, referee rules and inter-vehicle thresholds remain unchanged

TABLE VIII

HOLOOCEAN CLEAN/CURRENT VALIDATION AND TWO-ROVER SMOKE DEMONSTRATION OVER FIVE SEEDS. GATES AND GATE/WORLD COLLISIONS ARE MEAN $\pm$ STANDARD DEVIATION OVER ALL SEEDS. TIME IS THE OFFICIAL FIRST-TO-LAST-GATE TIME OVER FINISHED RUNS ONLY (TEAM: ELAPSED).

Setting	Controller / track	Seeds	Status	Gates	Time (s)	Collisions
Clean	rule_gate_baseline, Horseshoe Bay	5	5/5 FIN	12/12	232.4 ± 7.5	0.6 ± 0.5
	rule_gate_baseline, Vertical Serpent	5	5/5 FIN	17/17	259.7 ± 27.8	0.2 ± 0.4
	rule_gate_baseline, Mixed Endurance	5	5/5 FIN	22/22	540.6 ± 7.7	0.8 ± 0.8
Currents	rule_gate_baseline, Horseshoe, medium	5	3/5 FIN	$10.8 \pm 1.5/12$	315.0 ± 42.3	82.6 ± 74.1
	rule_gate_baseline, Horseshoe, strong	5	0/5 FIN	$3.0 \pm 0.0/12$	—	11.0 ± 1.1
Fleet	bluerov2_01, Horseshoe Bay	5	5/5 FIN	12/12	230.7 ± 5.4	1.4 ± 0.5
	bluerov2_02, Horseshoe Bay	5	5/5 FIN	12/12	236.5 ± 1.4	1.2 ± 0.4
	team aggregate	5	5/5 FIN	24/24	340.7 ± 1.4 (elapsed)	2.6 ± 0.8

Penalized time of (17), reported in the text where it differs from the official time: 428.3 ± 98.5 s on Horseshoe Bay under the medium current (finished seeds) and 353.7 ± 3.2 s for the team aggregate.

TABLE IX

FLEET COORDINATION ON THE DETERMINISTIC KINEMATIC SUBSTRATE (HORSESHOE BAY, OFFICIAL MODE, PENALIZE MODE FOR INTER-VEHICLE PROXIMITY EVENTS). A STAGGERED HETEROGENEOUS TEAM HAS A SLOWER SMOOTH_GATE_BASELINE LEADER AND FASTER ACOUSTIC_BASELINE FOLLOWERS. EACH TEAM SIZE IS RUN WITH AND WITHOUT LEADER-FOLLOWER COORDINATION. EVERY ROVER FINISHES IN EVERY CONDITION.

Team	Condition	Inter-vehicle $\downarrow$	Team penalized (s)
3	no coordination	2	216.1
	leader-follower	0	251.1
4	no coordination	3	221.9
	leader-follower	0	274.1
5	no coordination	4	226.8
	leader-follower	0	295.5

(R4). The results are exactly reproducible and isolate the control logic. Inter-vehicle proximity events represent geometric proximity on this substrate, not HoloOcean contact dynamics.

The two controllers occupy different operating points. Both finish all 12 gates. `acoustic_baseline` takes 120.1s, whereas the conservative `smooth_gate_baseline` takes 198.0s, or $1.6\times$ as long. The mean per-step change in surge and yaw commands is used as a lower-is-smoother proxy. It drops from 0.0159 to 0.0035, a reduction of roughly $4.5\times$. The benchmark therefore distinguishes the two legal controllers by completion time and behavior.

The fleet comparison uses a slower `smooth_gate_baseline` leader with faster `acoustic_baseline` followers. The start gap is 8s, lateral spacing is 1.5m and scoring uses `penalize` mode. Without coordination, the followers overtake the leader and trigger the proximity detector. Teams of three, four and five record 2, 3 and 4 inter-vehicle proximity events, respectively (Table IX). Leader-follower coordination removes all of these events, matching the single-rover result, while every rover finishes the full gate sequence. Team time increases because the followers pace behind the leader instead of overtaking it. This is the measured safety-throughput tradeoff.

The two-gate yield margin of (22) is necessary. With a one-gate margin ($\Delta g = 1$), a follower can reach a gate just as its predecessor clears it, so a full gate of physical spacing is not preserved. The four-rover team then records 6 inter-vehicle proximity events, twice the uncoordinated team's 3. With $\Delta g = 2$, it records none. The coordination also tolerates channel loss. Repeating the four-rover coordinated run with 20% per-link packet loss still yields no inter-vehicle proximity events and every rover finishes. Periodic heartbeats and the freshness window absorb the dropped packets. These results support the separation argument of Section VII-D: a two-gate progress lead becomes physical spacing. The deterministic kinematic substrate isolates this proximity logic but does not model contact. The next subsection tests the same policy in HoloOcean.

F. HoloOcean Diagnostic Validation

HoloOcean diagnostic validation tests the same coordination under contact dynamics. It uses only the clean Horseshoe Bay track in official mode with three rovers, no currents or obstacles, two start gaps (8.0 and 0.0s) and three seeds. The team has a `smooth_gate_baseline` leader and `acoustic_baseline` followers. Inter-vehicle mode is diagnostic and no parameters are retuned. All six coordinated runs finish with 36/36 team gates. They record no inter-vehicle proximity events, gate/world collisions, out-of-bounds events or stuck events. Penalized time ranges from 130.7–131.3s (Table X). Without coordination, faster followers crowd the slower leader. These runs record one or two inter-vehicle proximity events and up to 531 gate/world collisions. In two of the six runs, both with the 8s gap, the leader remains unfinished at 4/12 gates.

Contact dynamics change the tradeoff observed on the deterministic kinematic substrate. There, coordination removes inter-vehicle proximity events at a small time cost. In HoloOcean, the same crowding produces costly gate/world collisions, so coordination becomes a net gain. Finished uncoordinated runs are slightly faster in raw time (104–118s versus ~ 131 s when coordinated). Once gate/world collisions are scored, their penalized times rise to 222–404s. Two of

TABLE X

HOLOOCEAN DIAGNOSTIC VALIDATION ON CLEAN HORSESHOE BAY WITH THREE ROVERS, OFFICIAL MODE, NO CURRENTS OR OBSTACLES, DIAGNOSTIC INTER-VEHICLE MODE, TWO START GAPS, THREE SEEDS AND NO PARAMETER TUNING. THE TEAM HAS A `SMOOTH_GATE_BASELINE` LEADER AND `ACOUSTIC_BASELINE` FOLLOWERS. EACH ROW AGGREGATES THREE SEEDS AND RANGES SPAN THEM. FINISHED COUNTS SEEDS IN WHICH EVERY ROVER COMPLETES THE COURSE; IV COUNTS INTER-VEHICLE PROXIMITY EVENTS; COLL. COUNTS GATE/WORLD COLLISIONS. NO RUN RECORDS AN OUT-OF-BOUNDS OR STUCK EVENT.

Gap (s)	Condition	Finished	IV ↓	Coll. ↓	Penalized (s) ↓
8.0	no coordination	1/3	2	21–531	222.8 [†]
	leader-follower	3/3	0	0	130.7–131.3
0.0	no coordination	3/3	1	28–60	244.4–404.3
	leader-follower	3/3	0	0	130.7–130.8

[†] Over the one finished seed; the other two left the leader unfinished at 4/12 gates.

the six uncoordinated runs do not finish. This HoloOcean diagnostic validation is deliberately limited to one clean track, three rovers, two start gaps and three seeds. Extending it to other tracks, currents, obstacles and larger fleets remains future work (Section IX).

G. Reproducibility

Every run emits a structured event log and summary. The repository provides the exact commands used here [18]. A single configuration-driven entry point and per-track JSON files make each official run reproducible from one command. The fleet demonstration and unit-test suite are also scripted, so Table VIII can be regenerated end to end. One engine-free command produces the deterministic comparison in Table IX bit-for-bit.

IX. FUTURE WORK

We see four concrete next steps.

(i) *Cooperative fleet strategies beyond leader-follower conveying.* The leader-follower coordination of Section VII-D has already been evaluated on the deterministic kinematic substrate. It has also undergone HoloOcean diagnostic validation on clean Horseshoe Bay (Section VIII-F). The immediate next step is to extend that diagnostic validation to other tracks, currents, obstacles and larger fleets. Later work can examine formation keeping, cooperative overtaking, dynamic role assignment and robustness to uncertain relative localization or heavier acoustic loss. Such strategies must respect the low bandwidth and high latency of acoustic communication [15] described in the AUV-formation literature [16], [17].

(ii) *Competitive multi-team and heterogeneous-rover evaluation.* Extend the cooperative fleet setting to multiple rover teams or rover families with different dynamics and sensing suites. They would compete under the same referee and observation contract. This extension requires team-isolated scoring, ranking across teams, controlled interaction policies and normalization rules that compare heterogeneous platforms without hiding their physical differences.

(iii) *Current-compensation track family.* Promote the analytic current field of (6) to (8) into a graded official track family based on the provided `medium` and `strong` profiles. Controllers can then be developed to reject these currents. One legal direction is a cascaded design with an inner loop that serves DVL-measured body velocity to the guidance setpoint. This loop would reject the current as a disturbance without reading the true current vector, beneath outer beacon and vision guidance.

(iv) *Learning-based controllers.* The official observation contract can serve directly as a reinforcement-learning interface. The state contains beacon range, bearing and elevation; depth; IMU; DVL and velocity; and camera data. The normalized command of (3) is the action. A reward can combine sequential gate progress with the time and collision penalties of (17). A model-free policy-gradient or actor-critic method could train at scale on the deterministic kinematic adapter and then be fine-tuned in HoloOcean. This approach follows work on champion-level drone racing against a simulated opponent [5] and GPU-parallel robot-learning environments [13], [14] without weakening the official observation contract.

X. CONCLUSION

This paper presented Marine Race Arena, a configurable HoloOcean benchmark layer for underwater gate racing and team-level fleet evaluation. The framework strictly separates autonomy from evaluation. A controller implements three methods and sees only the documented official observation. An independent referee uses privileged state to validate gate crossings, penalties, timeouts, ranking and team scoring. We formalized the benchmark instance, gate-crossing geometry, single-rover and team scoring and staggered fleet semantics, then grounded them in the implementation.

In HoloOcean clean validation, the deterministic rule baseline completes all three official tracks in each of five seeds under the official observation contract. The staggered two-rover smoke demonstration also finishes in every seed with no inter-vehicle proximity events. Under currents, the same baseline degrades sharply: the `medium` profile finishes in only 3/5 seeds and the `strong` profile finishes in none. These failures remain part of the benchmark evidence because gates, margins and current profiles are unchanged.

On the deterministic kinematic substrate, the benchmark separates two legal controllers by completion time and motion smoothness. Leader-follower coordination uses only the optional acoustic channel and reduces a staggered team’s inter-vehicle proximity events to the single-rover level while every rover finishes. HoloOcean diagnostic validation on clean Horseshoe Bay covers three rovers, no currents or obstacles, two start gaps and three seeds. All six coordinated runs finish with no inter-vehicle proximity events or gate/world collisions. Uncoordinated teams incur many gate/world collisions and sometimes fail to finish.

REFERENCES

- [1] J. Betz, H. Zheng, A. Liniger, U. Rosolia, P. Karle, M. Behl, V. Krovi, and R. Mangharam, “Autonomous vehicles on the edge: A survey on

autonomous vehicle racing,” *IEEE Open Journal of Intelligent Transportation Systems*, vol. 3, pp. 458–488, 2022.

- [2] M. O’Kelly, H. Zheng, D. Karthik, and R. Mangharam, “F1TENTH: An open-source evaluation environment for continuous control and reinforcement learning,” in *Proceedings of the NeurIPS 2019 Competition and Demonstration Track*, ser. Proceedings of Machine Learning Research, vol. 123, 2020, pp. 77–89. [Online]. Available: <https://proceedings.mlr.press/v123/o-kelly20a.html>
- [3] S. Hoffmann, S. Sagmeister, T. Betz, J. Bongard, S. Büttner, D. Ebner, D. Esser, G. Jank, S. Goblirsch, A. Langmann, M. Leitenstern, L. Ögretmen, P. Pitschi, A.-K. Schwehn, C. Schröder, M. Weinmann, F. Werner, B. Lohmann, J. Betz, and M. Lienkamp, “Head-to-head autonomous racing at the limits of handling in the A2RL challenge,” *arXiv preprint arXiv:2602.08571*, 2026, abu Dhabi Autonomous Racing League; preprint, not peer reviewed. [Online]. Available: <https://arxiv.org/abs/2602.08571>
- [4] P. Foehn, D. Brescianini, E. Kaufmann, T. Cieslewski, M. Gehrig, M. Muglikar, and D. Scaramuzza, “AlphaPilot: Autonomous drone racing,” in *Robotics: Science and Systems (RSS)*, 2020. [Online]. Available: <https://www.roboticsproceedings.org/rss16/p081.html>
- [5] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, “Champion-level drone racing using deep reinforcement learning,” *Nature*, vol. 620, no. 7976, pp. 982–987, 2023.
- [6] E. Potokar, S. Ashford, M. Kaess, and J. G. Mangelson, “HoloOcean: An underwater robotics simulator,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2022, pp. 3040–3046. [Online]. Available: <https://www.ri.cmu.edu/publications/holocean-an-underwater-robotics-simulator/>
- [7] M. von Benzon, F. F. Sørensen, E. Uth, J. Jouffroy, J. Liniger, and S. Pedersen, “An open-source benchmark simulator: Control of a BlueROV2 underwater robot,” *Journal of Marine Science and Engineering*, vol. 10, no. 12, p. 1898, 2022.
- [8] M. M. M. Manhães, S. A. Scherer, M. Voss, L. R. Douat, and T. Rauschenbach, “UUV simulator: A Gazebo-based package for underwater intervention and multi-robot simulation,” in *OCEANS 2016 MTS/IEEE Monterey*, 2016, pp. 1–8.
- [9] P. Cieślak, “Stonefish: An advanced open-source simulation tool designed for marine robotics, with a ROS interface,” in *OCEANS 2019 - Marseille*, 2019, pp. 1–6.
- [10] M. M. Zhang, W.-S. Choi, J. Herman, D. Davis, C. Vogt, M. McCarrin, Y. Vijay, D. Dutia, W. Lew, S. Peters, and B. Bingham, “DAVE aquatic virtual environment: Toward a general underwater robotics simulator,” in *Proceedings of the IEEE/OES Autonomous Underwater Vehicles Symposium (AUV)*, 2022, pp. 1–8. [Online]. Available: <https://arxiv.org/abs/2209.02862>
- [11] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, “CARLA: An open urban driving simulator,” in *Proceedings of the Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 78, 2017, pp. 1–16. [Online]. Available: <https://proceedings.mlr.press/v78/dosovitskiy17a.html>
- [12] Autonomous Robotics Research Center, Technology Innovation Institute, “A2RL: Abu Dhabi autonomous racing league,” <https://a2rl.io>, 2024, autonomous car and drone racing championship; accessed 2026-06-27.
- [13] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, “OpenAI gym,” *arXiv preprint arXiv:1606.01540*, 2016. [Online]. Available: <https://arxiv.org/abs/1606.01540>
- [14] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State, “Isaac gym: High performance GPU-based physics simulation for robot learning,” in *Proceedings of the NeurIPS 2021 Track on Datasets and Benchmarks*, 2021. [Online]. Available: <https://arxiv.org/abs/2108.10470>
- [15] M. Stojanovic and J. Preisig, “Underwater acoustic communication channels: Propagation models and statistical characterization,” *IEEE Communications Magazine*, vol. 47, no. 1, pp. 84–89, 2009.
- [16] Y. Yang, Y. Xiao, and T. Li, “A survey of autonomous underwater vehicle formation: Performance, formation control, and communication capability,” *IEEE Communications Surveys & Tutorials*, vol. 23, no. 2, pp. 815–841, 2021.
- [17] T. Yan, Z. Xu, and S. A. Gadsden, “Formation control of multiple autonomous underwater vehicles: A review,” *Intelligence & Robotics*, vol. 3, no. 1, pp. 1–22, 2023.
- [18] A. Bedei, “HoloDroneCompetition: Marine race arena repository,” <https://github.com/AndreaBedei/HoloDroneCompetition>, 2026, v0.1 release candidate, accessed 2026-06-27.